
Projet de Développement – RT809

Application de Covoiturage Temps Réel (Jakarta EE)

Auteur :

Rémi Rahir / Moussa Tayeb Nemiche

Master 1 - RT809

20 juin 2026

Table des matières

1	Introduction	3
2	Cahier des Charges Détaillé	3
2.1	Contexte et Objectifs Principaux	3
2.2	Typologie des Acteurs du Système	3
2.3	Périmètre Fonctionnel et Règles de Gestion	3
2.4	Exigences Non-Fonctionnelles et Critères d'Acceptation	4
2.5	Gestion des Erreurs et Cas Limites	4
3	Architecture Technique	5
4	Tour d'horizon des techniques existantes et choix technologiques	5
4.1	Choix de la plateforme : Jakarta EE vs Spring Boot	5
4.2	Choix de l'Architecture API vs MVC Traditionnel	5
4.3	Gestion du Temps Réel : Polling AJAX vs WebSockets	6
5	Modélisation de la Base de Données	6
6	Mécanismes Détaillés et Fonctionnement	8
6.1	Phase 1 : Signalement Temps Réel (Passager)	8
6.2	Phase 2 : Dispatching et Prise en Charge (Conducteur)	9
6.3	Phase 3 : Clôture du Trajet et Déverrouillage des Notes	9
6.4	Phase 4 : Système de Notation Mutuelle	9
7	Analyse Détaillée du Code Source et Implémentation	9
7.1	Configuration du Projet et Serveur d'Application	9
7.2	Couche Modèle : Persistance avec JPA	10
7.3	Couche d'Accès aux Données (DAO - Data Access Object)	11
7.4	Couche Contrôleur : Servlets et Injection de Dépendances	11
7.5	Couche Vue : JSPs, GPS et Polling AJAX	11
7.6	API JSON Temps Réel	12
8	Conclusion	12

1 Introduction

Dans le cadre de l'unité d'enseignement RT809, nous avons développé une application Web de covoiturage basée sur la plate-forme **Jakarta EE**. Contrairement aux applications classiques de réservation à l'avance, ce système repose sur un mécanisme de signalement en *temps réel* depuis des points de rendez-vous spécifiques.

Les passagers se rendent à un point de rendez-vous, envoient leur localisation (GPS ou sélection manuelle) et diffusent leur destination. Les conducteurs de la même zone, alertés en direct via polling AJAX sur leur tableau de bord, peuvent se rendre sur place pour les prendre en charge après un échange de code d'identification. À l'issue du trajet, un système de notation mutuelle et un historique des trajets sont disponibles.

2 Cahier des Charges Détaillé

2.1 Contexte et Objectifs Principaux

L'idée de départ était de concevoir une application Web de covoiturage qui sorte un peu du schéma classique, en mettant en relation conducteurs et passagers de façon instantanée sur des lignes déjà établies. Là où des plateformes comme BlaBlaCar misent tout sur la réservation à l'avance, on a préféré partir sur un modèle de covoiturage spontané, géré en temps réel. Concrètement, le but est de fluidifier les déplacements du quotidien, en particulier les trajets domicile-travail ou les liaisons interurbaines récurrentes, en raccourcissant l'attente aux points désignés et en évitant complètement l'étape de réservation.

2.2 Typologie des Acteurs du Système

Le système cible et gère l'interaction entre trois profils d'utilisateurs distincts :

- **Le Passager** : Utilisateur se trouvant physiquement à un point de rendez-vous. Il utilise l'application pour signaler sa présence (via GPS ou sélection manuelle), indiquer sa destination souhaitée et obtenir un code de sécurité généré pour le trajet.
- **Le Conducteur** : c'est l'utilisateur qui a une voiture et un nombre de places limité à proposer. Son tableau de bord se rafraîchit en temps réel grâce au polling AJAX, et lui montre les passagers qui attendent dans sa zone de couverture.
- **Le Double Profil (les_deux)** : un utilisateur qui peut basculer entre les deux modes, passager ou conducteur, via le paramètre `force_role` stocké en session.
- **L'Administrateur (Mode Supervision)** : ce profil dispose d'un accès plus large pour superviser la plateforme : il gère la liste des points de rendez-vous, modère les notes ou commentaires abusifs et veille au bon fonctionnement global. Cette partie reste pour l'instant pensée pour de futures évolutions.

2.3 Périmètre Fonctionnel et Règles de Gestion

Le périmètre d'implémentation obligatoire du socle technique couvre les exigences (Features) suivantes :

1. **Authentification et Comptes** : Inscription et connexion sécurisées. Chaque utilisateur détient un compte unique (rôle *passager*, *conducteur* ou *les_deux*) qui conserve son historique et son score de réputation.

2. **Référentiel des Lieux** : Le système propose une liste en base de données des *Points de rendez-vous* disponibles, chacun géolocalisé (latitude/longitude) et rattaché à une *zone*.
3. **Localisation du Passager** : Le passager peut utiliser l'API `navigator.geolocation` du navigateur ; le serveur rattache alors automatiquement le signalement au point de RDV le plus proche. Un fallback manuel reste disponible.
4. **Création du Signalement** : Processus simplifié pour émettre une alerte « Je suis ici, je vais là ». Cette alerte est diffusée aux conducteurs de la zone concernée.
5. **Traitement Temps Réel (*Broadcasting*)** : L'affichage des alertes côté conducteurs et la notification de prise en charge côté passagers se font sans rechargement manuel, via polling AJAX (`/api/alertes` et `/api/statut-passager`).
6. **Gestion des Places** : à l'inscription, le conducteur indique combien de places il propose. Ce compteur baisse à chaque prise en charge et remonte automatiquement une fois le trajet terminé.
7. **Mécanisme de Mise en Relation Sécurisée (*Handshake*)** : pour que la prise en charge démarre, le conducteur doit saisir le `codeValidation` à 4 chiffres que possède le passager.
8. **Cycle de vie du Trajet** : une instance de trajet persistante est créée dès la prise en charge. C'est ensuite au passager de certifier son arrivée à destination, ce qui marque la course comme terminée.
9. **Gestion des Profils et Confiance** : Notation réciproque (1 à 5 étoiles + commentaire) une fois le trajet achevé.
10. **Historique** : Chaque utilisateur consulte ses trajets passés (date, point de RDV, destination, partenaire, notes données/reçues).

2.4 Exigences Non-Fonctionnelles et Critères d'Acceptation

Afin d'assurer le sérieux de la plateforme, plusieurs barrières logiques doivent être imposées :

- **Unicité des États** : Un passager ne doit pouvoir émettre qu'un seul et unique signalement à l'instant T. Le système doit bloquer les requêtes multiples pour éviter le spam.
- **Intégrité des Données** : Le processus d'évaluation n'est déclenchable que si la variable d'état du trajet est strictement certifiée comme "terminée". Une évaluation ne peut être soumise qu'une unique fois par acteur pour un même trajet.
- **Simplicité d'Utilisation (UX)** : L'interface doit être épurée et explicite. L'utilisateur doit savoir à tout instant s'il est « *en attente* », « *pris en charge* » ou « *en trajet* ». Les messages flash (`?msg=` / `?err=`) couplés aux mises à jour AJAX donnent ce retour visuel en continu.
- **Robustesse Technique** : on s'appuie sur les contraintes JPA (clés étrangères, unicité de l'email) et sur une gestion explicite des erreurs côté serveur, que ce soit un code incorrect, des places épuisées ou un signalement déjà pris, avec à chaque fois une redirection et un message clair pour l'utilisateur.

2.5 Gestion des Erreurs et Cas Limites

Le tableau suivant recense les principaux scénarios d'erreur traités par l'application :

- **Signalement déjà actif** : si un passager a déjà un signalement en cours, `CreerSignalementServ` refuse d'en créer un second.
- **Code incorrect** : quand le code saisi ne correspond pas, `DemarrerTrajetServlet` redirige en affichant un message d'erreur.
- **Places épuisées** : prise en charge refusée si `placesDisponibles` ≤ 0 .
- **Signalement déjà pris** : tentative de démarrage sur un signalement non *en attente* rejetée.
- **Notation duplicate** : `NotationDao.aNoteTrajet` empêche une double évaluation sur le même trajet.
- **Rôle inadapté** : pour le profil *les_deux*, les servlets vérifient systématiquement le `roleActif` en session (mode passager ou conducteur).
- **Session expirée** : redirection vers `/connexion` si l'utilisateur n'est pas authentifié.

3 Architecture Technique

L'application repose sur l'architecture classique MVC (Modèle-Vue-Contrôleur) avec les technologies suivantes :

- **Langage** : Java 17
- **Serveur d'application** : WildFly (version 5.0.0.Final via Maven plugin)
- **Persistance** : Hibernate ORM avec une base de données en mémoire (H2)
- **Backend (Contrôleurs & DAO)** : Servlets Jakarta EE, EJB (`@Stateless`), JPA
- **Frontend (Vues)** : JSP, JavaScript (GPS, polling AJAX), CSS personnalisé

4 Tour d'horizon des techniques existantes et choix technologiques

Dans le cadre de la construction d'une application transactionnelle et concurrente comme le covoiturage en temps réel, plusieurs approches d'ingénierie s'opposaient :

4.1 Choix de la plateforme : Jakarta EE vs Spring Boot

Aujourd'hui, l'écosystème Java tourne très largement autour de Spring Boot, qui mise sur une configuration rapide et des serveurs embarqués comme Tomcat ou Jetty. On a pourtant choisi de rester sur **Jakarta EE** (anciennement Java EE), volontairement. Ce choix tient avant tout à une volonté pédagogique : maîtriser les standards de l'industrie (JPA, Servlets, EJB) directement, sans la couche d'abstraction que Spring ajoute par-dessus. L'utilisation d'un vrai serveur d'application comme **WildFly** offre une robustesse native pour le pooling de la base de données et la gestion du cycle de vie transactionnel via les `@Stateless` EJBs, déléguant ainsi toute la lourdeur concurrente au conteneur.

4.2 Choix de l'Architecture API vs MVC Traditionnel

La conception moderne tend vers les architectures *décousues* : un backend API *RESTful* strict dialoguant avec un frontend *Single Page Application* (React, Angular). Pour ce projet, le modèle **MVC avec rendu Serveur (JSP)** a été privilégié :

1. Il réduit la complexité du déploiement (un seul WAR monolithique généré par Maven).
2. Il encapsule nativement les sessions utilisateurs via l'API `HttpSession`.
3. L'injection via les `requestDispatcher` et les attributs JSP permet un chaînage sécurisé (*Pattern PRG*) de l'information sensible sans l'exposer au client.

4.3 Gestion du Temps Réel : Polling AJAX vs WebSockets

Le cœur de l'application repose sur la diffusion *en direct* des signalements. On avait deux options sur la table :

- **WebSockets (JSR 356)** : Canal bidirectionnel permanent entre le serveur et les clients. Solution optimale à grande échelle, mais plus complexe à déployer et à maintenir dans le cadre du TP.
- **Polling AJAX** : Le client interroge périodiquement des servlets JSON dédiés (`AlertesAjaxServlet`, `PassagerStatutAjaxServlet`) via `fetch()`.

On a finalement opté pour le **polling AJAX** (toutes les 3 secondes). Ce choix collait bien avec l'architecture MVC/JSP déjà en place, sans avoir à tout repartir vers une SPA. Côté serveur, ça reste léger puisque les requêtes restent stateless, donc pas de connexion à maintenir ouverte en permanence comme ce serait le cas avec des WebSockets. On a aussi essayé de limiter au maximum les rafraîchissements inutiles côté front : le tableau du conducteur n'est reconstruit que quand la liste des alertes a réellement changé, ce qui évite d'effacer le code secret qu'il est peut-être en train de taper.

5 Modélisation de la Base de Données

On a structuré le modèle de données en gardant en tête les contraintes du temps réel. Hibernate (JPA) s'occupe ensuite de relier ces entités entre elles via des relations transactionnelles cohérentes.

- **Utilisateur** : représente aussi bien les passagers que les conducteurs, avec les champs `role`, `zone` et `placesDisponibles`.
- **PointRendezVous** : Lieux de rassemblement avec `nom`, `zone`, `latitude` et `longitude`.
- **Signalement** : c'est l'entité centrale de tout le processus. Elle démarre dans l'état "en attente", porte l'ID du passager, le point de RDV, la destination, et un `codeValidation` généré aléatoirement (4 caractères) qui sert à l'authentification physique lors de la prise en charge. Une fois ce code validé, elle bascule en "pris_en_charge".
- **Trajet** : Généré spontanément lorsque le conducteur valide le code secret d'un signalement. Il encapsule l'ID du conducteur, l'ID du passager et le statut du voyage ("*en_cours*" ou "*terminé*").
- **Notation** : Entité gérant le système d'évaluation post-trajet. Elle relie l'évaluateur, l'évalué et le **Trajet** d'origine tout en stockant une `note` entière sur 5 et un champ texte `commentaire`.

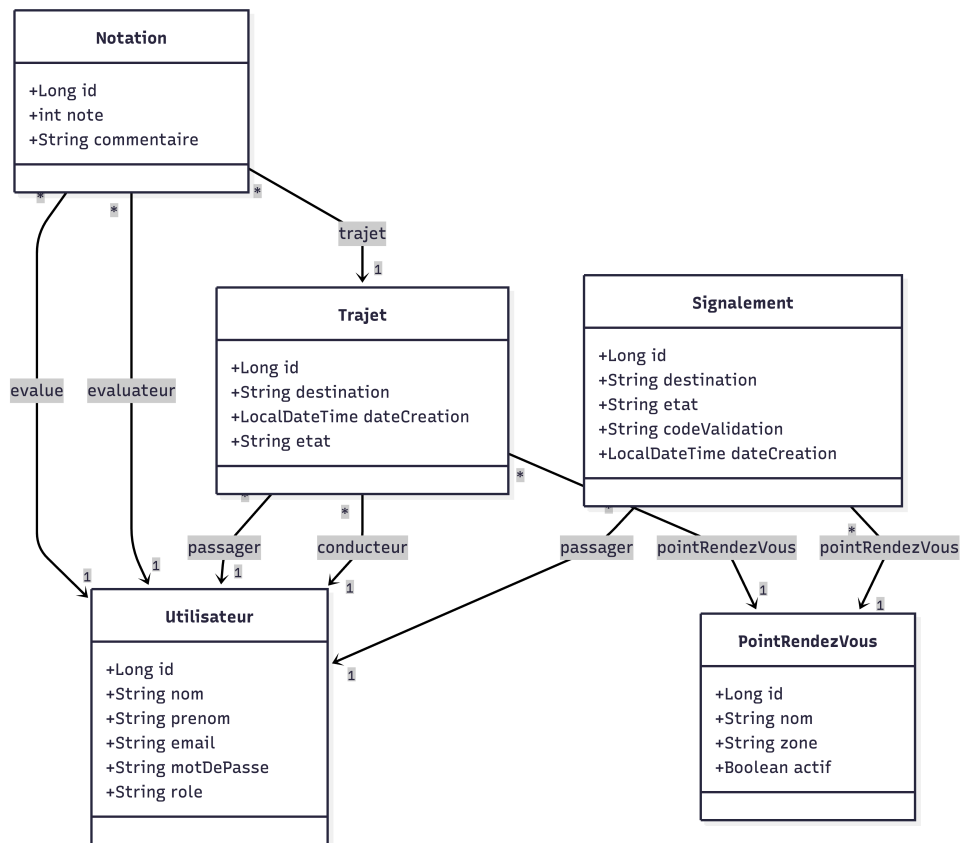


FIGURE 1 – Diagramme UML

6 Mécanismes Détaillés et Fonctionnement

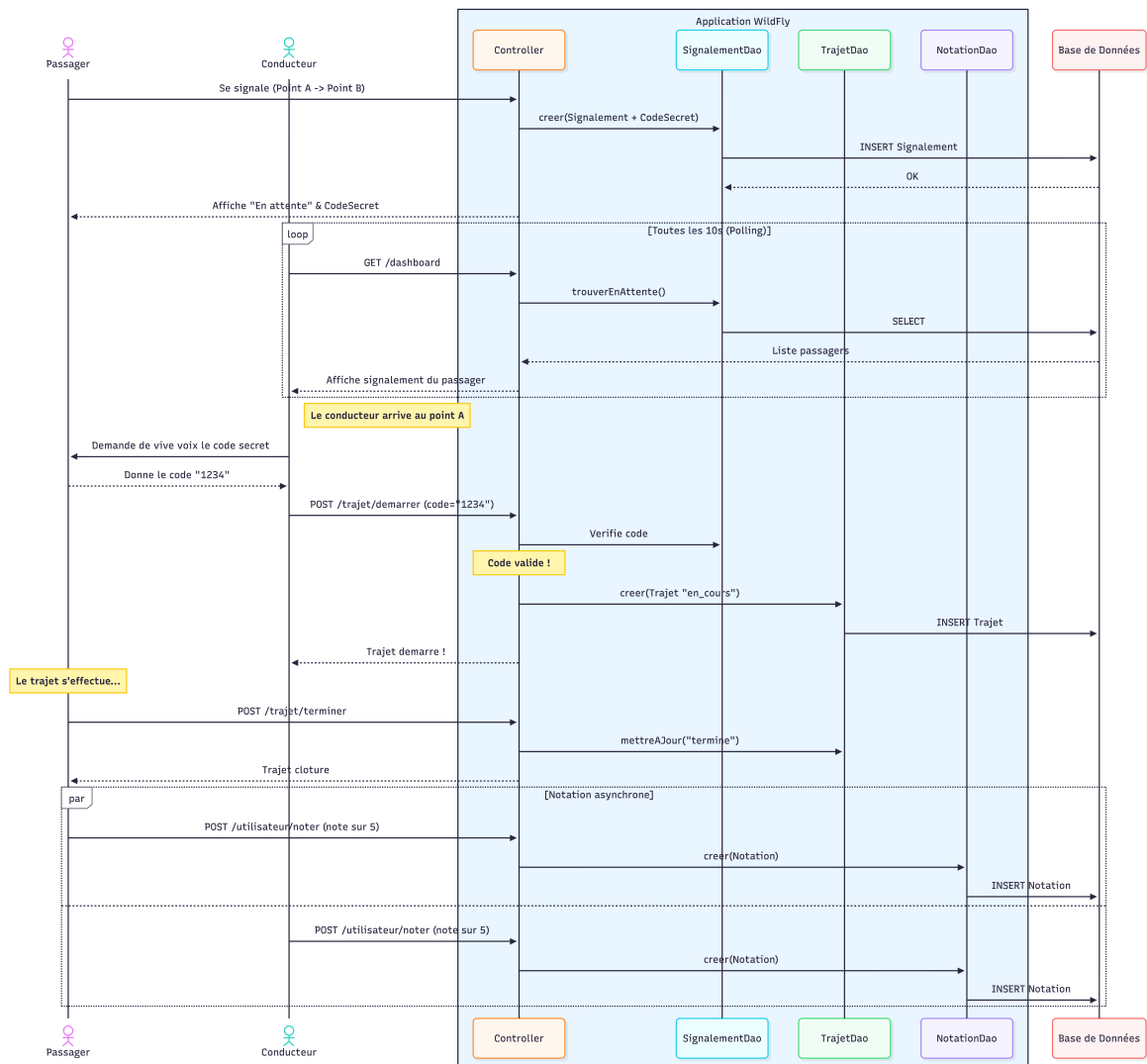


FIGURE 2 – Diagramme de Gestion des Changements

6.1 Phase 1 : Signalement Temps Réel (Passager)

Tout commence avec `CreerSignalementServlet`. Le passager a deux options pour se localiser : soit il utilise le bouton GPS (`navigator.geolocation`), et dans ce cas le client calcule lui-même le point de RDV le plus proche, met à jour le `<select>` en conséquence, puis envoie latitude et longitude au serveur qui confirme via `PointRendezVousDao.trouverPlusProche()` soit il choisit directement un point à la main si le GPS ne fonctionne pas ou n'est pas autorisé. Le backend génère un `codeValidation` à 4 chiffres, affiché sur le dashboard passager tant que le signalement est *en attente*. Un polling AJAX (`/api/statut-passager`) détecte ensuite la prise en charge sans rechargement manuel.

6.2 Phase 2 : Dispatching et Prise en Charge (Conducteur)

Côté conducteur, `AlertesAjaxServlet` (`/api/alertes`) retourne en JSON les signalements *en attente* filtrés par la zone du conducteur, ainsi que ses `placesRestantes`. Le dashboard reconstruit le tableau uniquement si la liste des alertes change, afin de ne pas effacer le code en cours de saisie. Lorsqu'un conducteur honore une demande :

1. Échange verbal du `codeValidation` avec le passager.
2. Saisie du code dans `DemarrerTrajetServlet`.
3. Vérification des places disponibles, validation du code, clôture du signalement (*pris_en_charge*), création du `Trajet` (*en_cours*) et décrémentation des places.

6.3 Phase 3 : Clôture du Trajet et Déverrouillage des Notes

Le passager certifie son arrivée via `TerminerTrajetServlet`. L'état du trajet passe à *terminé* et les places du conducteur sont réincrémentées. Le `DashboardServlet` affiche alors les formulaires de notation pour les trajets terminés non encore évalués, ainsi que la section *Mes trajets passés* (historique complet avec notes données et reçues).

6.4 Phase 4 : Système de Notation Mutuelle

Une fois l'indicateur d'état du trajet à *"terminé"*, le mécanisme d'évaluation prend le relais :

1. **Activation des Formulaires** : Le `DashboardServlet` effectue une sélection via filtre (une vérification d'existence `aNoteTrajet` injectée via `NotationDao`) pour identifier si l'utilisateur courant a déjà évalué son partenaire sur le trajet concerné.
2. **Soumission** : Si ce n'est pas le cas, un formulaire dynamique s'affiche incitant l'acteur (conducteur ou passager) à attribuer une note entre 1 et 5 et formuler un commentaire.
3. **Agrégation Globale** : À chaque ouverture de session ou actualisation, une requête SQL via JPA calcule la moyenne flottante de toutes les notes rattachées au profil de l'utilisateur (`calculerMoyenne`). Cette métrique est propagée universellement (ex. : *Note globale : 4.5/5 étoiles*) instaurant un mécanisme de réputation.

7 Analyse Détaillée du Code Source et Implémentation

Le volume de code produit (plusieurs classes Java, vues JSP, fichiers CSS et configurations XML) reflète la complexité inhérente à la mise en place d'un environnement Jakarta EE complet en partant de zéro. Cette section détaille des portions critiques du code source.

7.1 Configuration du Projet et Serveur d'Application

Le projet est géré par **Maven**, ce qui permet de standardiser la construction et de déployer directement l'application sur un conteneur **WildFly** embarqué via le plugin `wildfly-maven-plugin`.

```

1 <plugin>
2   <groupId>org.wildfly.plugins</groupId>
3   <artifactId>wildfly-maven-plugin</artifactId>
4   <version>5.0.0.Final</version>
5 </plugin>

```

Listing 1 – Extrait du pom.xml (plugin WildFly)

FIGURE 3 – Exemple de vue du tableau de bord passager

7.2 Couche Modèle : Persistance avec JPA

Les objets métiers sont mappés en base de données via Hibernate. L'utilisation d'annotations comme `@Entity` et `@PrePersist` permet d'automatiser les comportements, comme la gestion des timestamps de création des alertes.

```

1 @Entity
2 @Table(name = "signalements")
3 public class Signalement {
4     @Id
5     @GeneratedValue(strategy = GenerationType.IDENTITY)
6     private Long id;
7
8     @ManyToOne
9     @JoinColumn(name = "passager_id", nullable = false)
10    private Utilisateur passager;
11
12    @Column(nullable = false)
13    private String destination;
14
15    private String etat = "en attente";
16
17    @Column(length = 4)
18    private String codeValidation; // Code secret
19
20    @PrePersist
21    protected void onCreate() {
22        this.dateCreation = LocalDateTime.now();
23    }
24    // Getters et Setters...

```

25 }

Listing 2 – Extrait de l'Entité Signalement

7.3 Couche d'Accès aux Données (DAO - Data Access Object)

Les requêtes personnalisées (JPQL) sont rédigées dans des classes DAO, déclarées comme des composants `@Stateless` (Entreprise JavaBeans). Le conteneur se charge d'injecter l'`EntityManager` et de gérer la scalabilité transactionnelle.

```

1 public List<Signalement> trouverEnAttenteParZone(String zone) {
2     return em.createQuery(
3         "SELECT s FROM Signalement s WHERE s.etat = 'en attente' " +
4         "AND s.pointRendezVous.zone = :zone ORDER BY s.dateCreation DESC
5         ",
6         Signalement.class)
7         .setParameter("zone", zone)
8         .getResultList();
9 }

```

Listing 3 – Filtrage des alertes par zone (SignalementDao.java)

7.4 Couche Contrôleur : Servlets et Injection de Dépendances

La logique métier complexe (comme l'échange de clés) est opérée par les Servlets HTTP. Le conteneur injecte les beans EJB via `@Inject`. L'application respecte d'ailleurs le motif **PRG** (*Post/Redirect/Get*), bloquant la double soumission de formulaires en redirigeant systématiquement après chaque action POST.

```

1 conducteur = utilisateurDao.trouverParId(conducteur.getId());
2 Integer places = conducteur.getPlacesDisponibles();
3 if (places == null || places <= 0) {
4     response.sendRedirect(contextPath + "/dashboard?err=Aucune+place+
5     disponible.");
6     return;
7 }
8 // ...
9 if (alert.getCodeValidation().equals(codeSecret.trim())) {
10     alert.setEtat("pris_en_charge");
11     signalementDao.mettreAJour(alert);
12     Trajet t = new Trajet();
13     t.setConducteur(conducteur);
14     t.setPassager(alert.getPassager());
15     t.setEtat("en_cours");
16     trajetDao.creer(t);
17     conducteur.setPlacesDisponibles(places - 1);
18     utilisateurDao.mettreAJour(conducteur);
19 }

```

Listing 4 – Prise en charge avec verification du code et des places (DemarrerTrajetServlet.java)

7.5 Couche Vue : JSPs, GPS et Polling AJAX

Les vues JSP incluent la barre de navigation via `<jsp:include>`. Le dashboard passager intègre `navigator.geolocation` pour la localisation et un polling vers `/api/statut-passager`.

Le dashboard conducteur interroge `/api/alertes` toutes les 3 secondes :

```
1 function rafraichirAlertes() {
2     fetch(contextPath + "/api/alertes", { credentials: "same-origin" })
3     .then(function(response) { return response.json(); })
4     .then(function(data) {
5         document.getElementById("places-restantes").textContent =
6         data.placesRestantes;
7         if (idsAlertes(data.alertes) === dernieresAlertesIds) {
8             return; // pas de reconstruction si la liste est
9             identique
10         }
11         // reconstruction du tableau avec preservation des codes
12         saisis...
13     });
14 }
15 setInterval(rafraichirAlertes, 3000);
```

Listing 5 – Polling AJAX conducteur (dashboard_conducteur.jsp)

7.6 API JSON Temps Réel

Deux servlets dédiées exposent l'état métier en JSON :

- `AlertesAjaxServlet` (`/api/alertes`) : alertes filtrées par zone + places restantes (conducteur).
- `PassagerStatutAjaxServlet` (`/api/statut-passager`) : signalement en attente et trajet en cours (passager).

8 Conclusion

Au final, BlaBlaCovoit RT809 couvre bien ce qui était demandé dans le sujet de TP : un covoiturage spontané, en temps réel, construit sur Jakarta EE (WildFly, Maven, Git), avec signalement géolocalisé, filtrage par zone, gestion des places, échange de code entre passager et conducteur, un cycle de vie complet du trajet, et pour finir une notation mutuelle ainsi qu'un historique.

En prenant du recul, le polling AJAX plutôt que les WebSockets ou un simple rechargement de page nous semble avoir été un bon compromis entre réactivité et simplicité de mise en œuvre. Le profil *les_deux*, tout comme les deux servlets JSON ajoutées en cours de route, montrent qu'on s'est progressivement rapproché d'une architecture hybride entre MVC et API, sans pour autant abandonner le WAR monolithique, qui restait plus cohérent avec le cadre du module RT809.